DLDCA Inlab: Two Numbers Walk Into a Buffer

Week 6

September 9, 2026

Introduction

In this inlab, we build a program in five steps: a program that reads two non-negative integers from the console, adds them as 64-bit numbers, and prints the result back on the console.

We build this up step-by-step across 5 tasks. Note that you will have to copy code from earlier tasks into the later ones.

We will use the `read` and `write` syscalls, do integer division on x86 (which is weirder than you would expect), write a string-to-integer parser, use a given integer-to-string printer, and finally link against the C library and call `printf`.

Tasks 1-4 are purely assembly. Task 5 adds in calls to `printf`, a C function to make printing easier

Connecting to outlab 5

Parts of this lab overlap Outlab 5. This is done to keep the ideas flowing smoothly:

- **Outlab 5, Task 2a**: the `read` syscall and reading a line from stdin into a buffer.
- **Outlab 5, Task 1**: `my_atoi`, parsing a decimal integer out of a string.
- **Outlab 5, Task 3**: the read-then-write loop over a file descriptor.

Necessary tools

We use `nasm` to assemble, `ld` to link Tasks 1–4, and `gcc` to link Task 5 (which uses the C library). Check `nasm` and `gcc` are installed (`ld` ships with `gcc`):

```
$ nasm -v
NASM version [version]
$ gcc --version
gcc ([linux-flavour] [version]-[linux-build]) [version]
```

Every task is built and run the same way:

```
$ make [task]                # build [task] and run it
$ echo "25" | make [task]     # feed it input on stdin
```

Structure of submission/templates

Submission format:

```
your-roll-no/
|- labexam/
    |- raceclock.s (TODO)
|- task1/
    |- hello.s (TODO)
|- task2/
    |- divby10.s (TODO)
|- task3/
    |- atoi.s (TODO)
|- task4/
    |- itoa.s (TODO)
|- task5/
    |- adder.s (TODO)
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz` in lowercase. The command given below can be used to do the same

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Tasks

Lab Exam: It's Hammertime

A stopwatch counted `T` seconds. `T` is a single non-negative integer, given on standard input. Print the same duration as `H:MM:SS`, where

- `H` is the whole number of hours (it can be more than 24 — a stopwatch is not a wall clock),
- `MM` is minutes, always two digits,
- `SS` is seconds, always two digits.

Run from the `06_inlab/` directory:

```
$ echo "3661" | make labexam
1:01:01

$ echo "90061" | make labexam
25:01:01
```

What you are given

`raceclock.s` already contains a working `my_atoi` — do not edit it:

```
my_atoi(const char* buf{rdi}, size_t len{rsi}) -> uint64_t{rax}
```

It reads decimal digits from the front of `buf` and returns their value. It also has the format string ready:

```
fmt: db "%ld:%02ld:%02ld", 10, 0
```

You only need to write `_start`.

What `_start` must do

1. **Read the number.** Do one `read` syscall from stdin (fd `0`) into `buf`. `read` returns the byte count in `rax`. Call `my_atoi(buf, count)`; now `rax` holds `T`.
2. **Extract hours.** Divide `T` by `3600`. On x86 the `div` instruction divides `rdx:rax` by its operand and leaves the *quotient* in `rax` and the *remainder* in `rdx`. So:

   `H` = `T / 3600`, leftover seconds = `T % 3600`.
3. **Extract minutes and seconds.** Divide the leftover by `60`:

   `M` = leftover `/ 60`, `S` = leftover `% 60`.

   Both are between 0 and 59.
4. **Print.** Set up and make exactly this call:

   ```
   printf("%ld:%02ld:%02ld\n", H, M, S)
   ```

   That is: `rdi` = address of `fmt`, `rsi` = `H`, `rdx` = `M`, `rcx` = `S`, and `al` = `0` (`printf` is variadic; `xor eax, eax` sets the count of vector-register arguments). The `%02ld` in the format string does the zero-padding for you.
5. **Exit.** Call the libc `exit` with status `0` (`xor edi, edi` then `call exit`). Use the libc `exit`, not the raw syscall, so `printf`'s output is flushed.

Warning about `div`. `div` always uses the full 128-bit value `rdx:rax` as the dividend. If `rdx` is not `0`, you are dividing a garbage 128-bit number and the result is wrong (or the CPU faults). **Zero `rdx` with `xor edx, edx` right before every `div`.** The second division is easy to get wrong: after the first `div`, `rdx` still holds the remainder, so you must clear it again before the second one.

Building and testing

All commands run from the `06_inlab/` directory:

```
$ echo "3661" | make labexam    # build raceclock.s, run it once on that input
1:01:01
$ make labexam-check            # run every case in labexam/tests/
PASS -- labexam/tests/01_input.txt (0)
PASS -- labexam/tests/02_input.txt (1)
...
$ make clean
```

`make labexam` builds `raceclock.s` and runs it once on whatever you pipe in. `make labexam-check` runs it on every case in `labexam/tests/` (`N.in` with expected output `N.out`) — 0, the minute and hour boundaries, 23:59:59, past 24 hours, and a large value — and reports PASS or FAIL for each. Your job is a clean sweep of PASSes.

Task 1: Hello, it's me

Recall that syscalls are special function calls that you make to the OS. Each syscall has a number and a set of arguments. The "syscall number" is an integer in `rax`. The arguments are in registers `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`. Then you type `syscall` to make the call and voila! It does what you want. Return value is in `rax`.

In generic x86-64 code a syscall looks like this:

```
mov rax, <syscall number>
mov rdi, <arg 1>
mov rsi, <arg 2>
mov rdx, <arg 3>
:
syscall
; return value is now in rax
```

The `read` syscall (number `0`) takes input: it copies bytes from a file descriptor into a buffer you provide. A file descriptor (`fd`) is an unsigned integer which is an identifier for a file. The standard input, standard output and standard error are special streams treated as files with `fd` `0`, `1` and `2`. `read` takes `fd` in `rdi`, the buffer address in `rsi`, and the maximum number of bytes it can read in `rdx` (size of your buffer). It returns, in `rax`, how many bytes it actually read. That count includes the trailing newline when the user presses Enter.

Think: What minimum size should your buffer be, to read an unsigned 64-bit integer? (No leading zeros or sign.)

The `write` syscall (number `1`) is its mirror image. It takes the same three arguments, but it copies bytes *from* your buffer *to* a file descriptor. File descriptor `1` is standard output.

In `_start`: read one line from stdin into a buffer, then print `"Hello "` followed by exactly the bytes you read to stdout, using a two `write` syscalls. Exit with code `0`.

Think: Can you do it with one syscall? Note that one `write` sends one contiguous run of bytes, so `"Hello "` and the bytes you read have to sit next to each other in memory.

Hint. To repeat a byte multiple times in NASM, use the `times` directive together with `db`.

For example: `buffer` points to a buffer of 512 bytes each ontaining the value 0.

```
buffer: times 512 db 0
```

Fill in `_start` in `hello.s`.

```
$ echo "world" | make task1
Hello world

Return Code: 0
```

Task 2: Divide and conquer

There is no "divide register by register" instruction on x86. The `div` instruction always divides the 128-bit value held in the pair `rdx:rax` (`rdx` is the high half, `rax` the low half) by its single operand. Afterwards:

- `rax` holds the quotient $\lfloor (\texttt{rdx:rax}) \div (\texttt{reg/*mem}) \rfloor$
- `rdx` holds the remainder $(\texttt{rdx:rax}) \% (\texttt{reg/*mem})$

So to divide a plain 64-bit number by something, you put the number in `rax`, set the high half `rdx` to `0` (do `xor rdx, rdx`), put the divisor in another register, and `div` that register.

Note that zeroing is only correct for non-negative numbers. Signed division uses `idiv` and `cqo` to sign-extend `rax` into `rdx`. This lab only deals with non-negative numbers, so you will not need it.

Dividing by 10 in two steps. For any non-negative `n`,

$$\texttt{n}/10 \;=\; (\texttt{n}/2)\,/\,5$$

where every `/` is integer (floor) division. Dividing by 2 is just a right shift (`shr rax, 1`), no `div` needed. Dividing by 5 is one `div` with the `rdx` setup above. **This is just for practice. In later tasks, you will divide directly by 10.**

Task: Implement `div_by_10(uint64_t n{rdi}) -> uint64_t{rax}`

using a shift for dividing by `2` and a `div` for dividing by `5`. In `_start`, call `div_by_10(250)` and exit with the result.

Fill in `div_by_10` and `_start` in `divby10.s`.

```
$ make task2
Return Code: 25
```

Task 3: Mommy I can count!

Input arrives as text. `read` gives you the bytes `'2'`, `'5'`, `'\n'` — three characters, not the number 25. You need to turn the digit characters into an actual integer.

Step 1: ASCII codes. Every character is a byte whose value is called an ASCII doc.

```
'0' .. '9'   48 .. 57   (consecutive)
'\n'         10         newline, ends the line the user typed
' '          32         space
'\0'         0          null byte
```

The digits are *consecutive*: `'0'` is `48/0x30`, `'1'` is `49/0x31`, and so on up to `'9'` at `57/0x39`.

Step 2: character to digit value. Because the digit codes are consecutive, the numeric value of a digit character `c` is just `c - '0'`: `'7' - '0' = 55 - 48 = 7`.

Step 3: is this byte a digit? (home-style). The obvious test is two comparisons:

```
c >= '0'  AND  c <= '9'
```

Step 4: is this byte a digit? (bhavesh-style). Compute `d = c - '0'` first, as an *unsigned* byte. Then `c` is a digit exactly when

```
(uint8_t) d <= 9
```

Why one comparison is enough: if `c` is below `'0'` (say a newline, 10), then `c - '0'` is negative, and in unsigned 8-bit arithmetic, that wraps around to a large value near 255, comfortably `> 9`, so the test fails. If `c` is above `'9'`, then `d` is `> 9` directly. Only the ten real digits land in `0..9`. In assembly this is `sub` then a single `cmp` / `ja`.

Step 5: the loop. Iterate on the buffer left to right. Keep a running `result`. For each new digit, the digits already seen are have to shift one place to the left and new digit has to occupy one's place, so multiply `result` by 10 and add the new digit. Stop at the first byte that is not a digit. It could be the newline, or the end of whatever `read` gave you.

Step 6: pseudocode.

```
my_atoi(buf, len):
    result = 0
    for i in 0 .. len-1:
        d = buf[i] - '0'        # unsigned byte
        if d > 9:  # do unsigned comparison - use 'ja'
            break               # not a digit
        result = result * 10 + d
    return result
```

Task. Translate into `my_atoi(const char* buf{rdi}, size_t len{rsi}) -> uint64_t{rax}`

In `_start`: read a line from stdin, call `my_atoi` on it (the byte count from `read` is your `len`), and exit with the result.

Fill in `my_atoi` and `_start` in `atoi.s`.

```
$ echo "25" | make task3
Return Code: 25
```

Task 4: The adder (`my_itoa` is given)

Now let's put it together: read two numbers, add them, print the sum.

Going from number to text. Task 3 turned text into a number. Printing the sum needs the reverse: an integer to its decimal text, that is, `my_itoa`. It is harder to write than `my_atoi` for one reason: repeatedly dividing by 10 and taking the remainder gives you the digits *least significant first* (for 250 you get 0, then 5, then 2), so you have to reverse them before printing, and `n = 0` has to be special-cased to print a single `'0'`.

`my_itoa(uint64_t n{rdi}, char* buf{rsi}) -> size_t{rax}` is **given to you** in `itoa.s`. Read it once so you know what it does; you do not need to reproduce it.

It writes the decimal text of `n` into `buf` (no null terminator) and returns the number of digits written.

Reading two lines. One big `read` of 256 bytes would swallow *both* numbers at once when the input is piped in, leaving nothing for the second `read`. Read the first line one byte at a time and stop at the `'\n'`; then the next `read` starts on the second line. A helper that reads one line and returns its `my_atoi` value is worth writing once and calling twice.

Task. Copy your `my_atoi` from Task 3 into `itoa.s`. In `_start`:

1. Read the first line from stdin, parse it with `my_atoi`.
2. Read the second line, parse it.
3. Add them as 64-bit values.
4. `my_itoa` the sum into a buffer.
5. `write` the digits, plus a newline, to stdout.
6. Exit `0`.

Fill in `my_atoi` and `_start` in `itoa.s`.

```
$ printf '48\n18\n' | make task4
66

Return Code: 0
```

Task 5: Same adder, now with `printf`

Task 4 built the sum's digits by hand with `my_itoa` and pushed them out with a `write` syscall. This task hands both jobs to `printf`: give it the number and a format string, and it prints `"sum = 66"` for you. No more `my_itoa`. That convenience drags in the C library, so there is some setup.

Two libc calls. The C library (*libc*) is a normal library of compiled functions. This task uses two of them:

- `printf`: prints formatted output.
- `exit`: libc function that ends the program (it flushes `printf`'s buffered output first, then makes the exit syscall). **This is not the same as the `exit` syscall (number `60`) you have been calling directly.** `exit(int)` is a normal function here which you call with `call`, use it here to ensure that the result of `printf` is not lost.

Declare both at the top of the file:

```
extern printf
extern exit
```

Linking against libc. So far the makefile linked your object file on its own. `printf` and `exit` live in libc, so the linker has to link against it. The makefile now links this task with

```
gcc -no-pie -nostartfiles -o task5.out task5/adder.o
```

`-nostartfiles` means that the starting point is `_start`, otherwise one writes '`main`' in `C` and the linker sets up globals and then jumps to `main`, instead of using your `_start`.

`-no-pie` makes a plain, non-relocatable executable. Position-independent code and relocation are a topic of their own and we will *not* touch them here. Know that the addresses in your earlier `lea rsi, [buf]` instructions are filled in by the linker at a fixed location, and `-no-pie` keeps that working. (Otherwise we would have to use relative addresses as `[rel buf]`. You can read up on this on your own.)

Calling convention (SystemV ABI). Integer and pointer arguments go in `rdi, rsi, rdx, rcx, r8, r9`, in that order; the return value comes back in `rax`. `printf` is variadic, so it needs one extra thing: `al` must hold the number of arguments passed in vector (SSE) registers. You are passing none, so zero it (`xor eax, eax`).

Stack alignment. Before any `call` into libc, `rsp` must be 16-byte aligned, i.e. `rsp mod 16 == 0`. If it is not, you may have to drop some extra blank bytes (padding) onto the stack to make it so. One common way to force it at the top of `_start`:

```
    and rsp, -16
```

Here `-16` is the constant

```
    hex:     0xFFFF_FFFF_FFFF_FFF0
    binary:  1111....1111 0000     (60 ones, then four zeros)
```

`and`-ing `rsp` with this clears its low 4 bits, which rounds `rsp` *down* to the nearest multiple of 16. Rounding down means `rsp` moves to lower addresses — it grows the stack by up to 15 bytes, which is exactly the padding. It never rounds up. After that, **keep it aligned, i.e., always ensure 16-byte alignment whenever you do a** `call`.

Formatting. `printf`'s first argument is a format string; each `%` placeholder in it is filled by the next argument. `%ld` is the placeholder for a 64-bit signed integer (a C `long`). In C the call is:

```
    long sum = 66;
    printf("sum = %ld\n", sum);      // prints:  sum = 66
```

So you pass the sum itself to `printf` — no buffer, no null terminator, no digit conversion of your own.

Task. Copy `my_atoi` (Task 3) into `adder.s`, plus your two-line reading code. In `_start`:

1. Read and parse the two numbers, add them.
2. `printf("sum = %ld\n", sum)`.
3. `exit(0)` — the libc function.

Fill in `my_atoi` and `_start` in `adder.s`.

```
$ printf '48\n18\n' | make task5
sum = 66

Return Code: 0
```

Table 1: Some basic x86-64 Assembly Instructions

Instruction	Operands	Effect
mov	reg, reg/*mem/imm	reg ← reg/*mem/imm
mov	*mem, reg	*mem ← reg
movzx / movsx	reg, reg/*mem	reg ← reg/*mem, zero-/sign-extended
lea	reg, *mem	reg ← address of *mem
add	reg, reg/*mem/imm	reg ← reg + reg/*mem/imm
add	*mem, reg/imm	*mem ← *mem + reg/imm
sub	reg, reg/*mem/imm	reg ← reg − reg/*mem/imm
sub	*mem, reg/imm	*mem ← *mem − reg/imm
inc	reg/*mem	reg/*mem ← reg/*mem + 1
dec	reg/*mem	reg/*mem ← reg/*mem − 1
neg	reg/*mem	reg/*mem ← −reg/*mem (Two's complement)
not	reg/*mem	reg/*mem ← ¬reg/*mem (One's complement/bitwise)
shl / shr	reg/*mem, imm/cl	Logical shift left / right by imm or cl bits
sar	reg/*mem, imm/cl	Arithmetic shift right (preserves sign bit)
imul	reg, reg/*mem/imm	reg ← reg × reg/*mem/imm
imul	*mem, reg/imm	*mem ← *mem × reg/imm
div	reg	rax ← (rdx:rax)/reg rdx ← (rdx:rax)%reg
and	reg, reg/*mem/imm	reg ← reg ∧ reg/*mem/imm
and	*mem, reg/imm	*mem ← *mem ∧ reg/imm
or	reg, reg/*mem/imm	reg ← reg ∨ reg/*mem/imm
or	*mem, reg/imm	*mem ← *mem ∨ reg/imm
xor	reg, reg/*mem/imm	reg ← reg ⊕ reg/*mem/imm
xor	*mem, reg/imm	*mem ← *mem ⊕ reg/imm
cmp	reg/*mem, reg/imm	Sets flags based on subtraction: reg/*mem - reg/imm. ZF = 1 if equal, SF = 1 if result negative, CF = 1 if borrow (unsigned), OF = 1 if signed overflow.
test	reg/*mem, reg/imm	Sets flags based on bitwise AND: reg/*mem ∧ reg/imm. ZF = 1 if result is zero, SF = 1 if high bit set, CF and OF are cleared. No result is stored.
jmp	label	Unconditional jump to label
jmp	label	Unconditional jump
je / jne	label	Jump if (not) equal / zero
ja / jae	label	Jump if above / above-or-equal (unsigned)
jb / jbe	label	Jump if below / below-or-equal (unsigned)
jg / jge	label	Jump if greater / greater-or-equal (signed)
jl / jle	label	Jump if less / less-or-equal (signed)
cmov*cc*	reg, reg/*mem	reg ← reg/*mem if *cc* holds, else unchanged.
syscall		Performs a syscall with syscall number stored in rax Arguments passed in other registers starting from rdi
push	reg	Pushes the value in reg onto the stack
pop	reg	Pops the value on top of the stack onto reg
call	label	Calls the function called label
ret		Pops the return address from stack and jumps to it

Table 2: Syscalls used in this inlab (Tasks 1–4)

Syscall	`rax`	Arguments (`rdi, rsi, rdx`)	Returns (`rax`)
`read`	0	`fd, buf, count`	bytes read (0 = EOF)
`write`	1	`fd, buf, count`	bytes written
`exit`	60	`status`	does not return

A syscall is invoked with the `syscall` instruction: number in `rax`, arguments in `rdi, rsi, rdx, r10, r8, r9`, result back in `rax`.

Table 3: C library (libc) calls used in this inlab (Task 5)

Function	Arguments	Returns
`printf`	`fmt`, then the format's args	bytes written (`rax`)
`exit`	`status`	does not return

A libc function is invoked with the `call` instruction and follows the SystemV ABI: integer/pointer arguments in `rdi, rsi, rdx, rcx, r8, r9` (in that order), return value in `rax`, callee-saved registers `rbx, rbp, r12-r15`. Before the `call`, `rsp` must be 16-byte aligned; for a variadic function like `printf`, `al` must hold the number of vector-register arguments (0 here).

`exit` the syscall (number 60) and `exit` the libc function are different things. The syscall ends the process immediately. The libc function flushes `printf`'s buffered output first, then makes the exit syscall — use it in Task 5 so no output is lost.